

# AQI Prediction — Full ML Pipeline Report

ML Engineering Project | July 2026

---

title: "AQI Prediction — Full ML Pipeline Report" author: "ML Engineering Project"  
date: "July 2026"

---

## AQI Prediction — Full ML Pipeline Report

---

### 1. Objective

---

Build a comprehensive machine learning system that predicts the Air Quality Index (AQI) from pollutant concentrations across Indian cities, with per-city analysis, model interpretability (SHAP), future forecasting, an interactive dashboard, and automated retraining.

### 2. Dataset

---

- **Source:** Kaggle "Air Quality Data in India" ( `city_day.csv` )
- **Rows:** 29,531
- **Columns:** 16 (City, Date, PM2.5, PM10, NO, NO2, NOx, NH3, CO, SO2, O3, Benzene, Toluene, Xylene, AQI)
- **Time span:** 2015--2020
- **Cities:** 26 cities across India

## 3. Data Preprocessing & Feature Engineering

---

### 3.1 Imputation

Missing values are handled using **KNN Imputer** (k=5, applied per city), which preserves multivariate pollutant relationships better than simple mean imputation. Any remaining NaN is filled with the global column mean.

### 3.2 Feature Engineering

**Temporal features:** - Year, Month, Day, DayOfWeek, IsWeekend (binary) - Season (Winter/Spring/Summer/Autumn mapped from month)

**Interaction features (domain-driven):** -  $PM_{2.5} \times PM_{10}$ ,  $CO \times NO_2$ ,  $SO_2 \times NO_2$ ,  $PM_{2.5} \times CO$ ,  $O_3 \times NO_2$

**Ratio features:** -  $PM_{2.5} / PM_{10}$ ,  $NO_2 / NO_x$ ,  $CO / SO_2$

**City encoding:** - Target encoding: City  $\rightarrow$  mean AQI per city - Frequency encoding: City  $\rightarrow$  number of samples per city

### 3.3 Lag Features (for forecasting)

Lagged values (1, 2, 3, 7 days) for AQI,  $PM_{2.5}$ ,  $PM_{10}$ , CO,  $NO_2$ ,  $SO_2$ ,  $O_3$ . Rolling means (3-day, 7-day) for  $PM_{2.5}$ ,  $PM_{10}$ , CO, AQI. Rows with NaN from lag creation are dropped.

### 3.4 Scaling

StandardScaler (fit on training set only) applied to all numeric features.

## 4. Train/Test Split

---

Chronological split (time-series aware): **80% train / 20% test**, preserving temporal order. No shuffling.

- Train: Jan 2015 -- Aug 2019
- Test: Sep 2019 -- Jul 2020

## 5. Models & Hyperparameter Tuning

---

### 5.1 Models Tested

| Model                        | Tuning Method                 |
|------------------------------|-------------------------------|
| Linear Regression            | None (baseline)               |
| Ridge ( $\alpha=1.0, 10.0$ ) | Manual $\alpha$ values        |
| Lasso ( $\alpha=0.1, 1.0$ )  | Manual $\alpha$ values        |
| Random Forest                | RandomizedSearchCV (20 iters) |
| <b>XGBoost</b>               | RandomizedSearchCV (20 iters) |
| LightGBM                     | RandomizedSearchCV (20 iters) |
| CatBoost                     | RandomizedSearchCV (15 iters) |

### 5.2 Hyperparameter Search Spaces

**Random Forest:** - n\_estimators: 50, 100, 200, 300 - max\_depth: 5, 10, 15, 20, None - min\_samples\_split: 2, 5, 10 - min\_samples\_leaf: 1, 2, 4

**XGBoost:** - n\_estimators: 100, 200, 300 - max\_depth: 3, 5, 7, 10 - learning\_rate: 0.01, 0.05, 0.1, 0.2 - subsample: 0.6, 0.8, 1.0 - colsample\_bytree: 0.6, 0.8, 1.0

**LightGBM:** - n\_estimators: 100, 200, 300 - max\_depth: -1, 5, 10, 15 - learning\_rate: 0.01, 0.05, 0.1, 0.2 - num\_leaves: 31, 50, 100, 150 - subsample: 0.6, 0.8, 1.0

**CatBoost:** - iterations: 100, 200, 300 - depth: 4, 6, 8, 10 - learning\_rate: 0.01, 0.05, 0.1, 0.2 - l2\_leaf\_reg: 1, 3, 5, 10

### 5.3 Cross-Validation

TimeSeriesSplit (3-fold) to maintain temporal order during cross-validation.

## 6. Results

---

## 6.1 Model Performance (on test set)

Performance metrics will be populated after running the notebook. Expected ranking:

1. **XGBoost** (champion) — typically  $R^2 > 0.94$ , RMSE < 28
2. LightGBM
3. CatBoost
4. Random Forest
5. Ridge
6. Linear Regression
7. Lasso

Target thresholds:  $R^2 > 0.94$ , RMSE < 28, MAE < 19.

## 6.2 City-Specific vs Global Model

Per-city XGBoost models are trained for cities with  $\geq 50$  samples and compared against the global champion on each city's test split. The comparison answers: *does a specialized per-city model beat the one-size-fits-all global model?*

Results vary by city. The analysis shows which cities benefit most from customized models.

## 7. SHAP Interpretability

---

### 7.1 Global Feature Importance

SHAP TreeExplainer computes feature contributions for every prediction. The top features driving AQI are typically:

1. PM2.5 lag features and current values
2. PM10
3. CO
4. Temporal features (month, season)
5. Interaction features ( $\text{PM2.5} \times \text{PM10}$ )

### 7.2 Per-City SHAP Analysis

For major cities (Delhi, Mumbai, Kolkata, Chennai, Bangalore, Hyderabad, Pune, Ahmedabad), the top 3 most influential pollutants are identified, revealing city-specific pollution profiles.

## 8. Forecasting

---

A separate XGBoost forecasting model is trained using only lag, rolling, and temporal features (no current-day pollutants). This enables multi-step forecasting.

**Horizon evaluation:** 1-day, 3-day, and 7-day forecast quality is measured. Performance degrades with longer horizons as expected.

## 9. Interactive Dashboard

---

A Streamlit dashboard ( `dashboard/app.py` ) with three tabs:

### Tab 1: AQI Prediction + SHAP

- Input sliders for all 12 pollutants
- City selector
- Predict AQI, show category with color coding
- SHAP waterfall plot for explanation

### Tab 2: City Insights

- Per-city top pollutant table (from SHAP analysis)
- Feature reference guide

### Tab 3: 7-Day Forecast

- Recursive multi-step forecast using lag features
- Bar chart with AQI category colors
- Threshold lines (Good, Satisfactory, Moderate, Poor)

## 10. Automated Retraining

---

`scripts/retrain.py` runs the full pipeline end-to-end: 1. Load raw data 2. KNN imputation 3. Feature engineering 4. Train/test split 5. Model training + tuning (all 7 models) 6. Save champion + scaler + encoders 7. Train forecast model 8. Save processed datasets

Usage: `python scripts/retrain.py`

## 11. Artifacts

| File                                       | Description                   |
|--------------------------------------------|-------------------------------|
| <code>models/best_model.pkl</code>         | Champion ML model             |
| <code>models/forecast_model.pkl</code>     | Time-series forecast model    |
| <code>models/scaler.pkl</code>             | StandardScaler                |
| <code>models/scaler_forecast.pkl</code>    | Forecast StandardScaler       |
| <code>models/city_encoder.pkl</code>       | City target/frequency encoder |
| <code>models/feature_columns.pkl</code>    | Feature column names          |
| <code>models/forecast_features.pkl</code>  | Forecast feature names        |
| <code>models/city_shap_analysis.pkl</code> | Per-city SHAP results         |
| <code>data/processed/train.csv</code>      | Training set                  |
| <code>data/processed/test.csv</code>       | Test set                      |
| <code>docs/figures/*.png</code>            | EDA & SHAP figures            |

## 12. AQI Bucket Reference

| Category | AQI Range |
|----------|-----------|
| Good     | 0--50     |

| Category     | AQI Range |
|--------------|-----------|
| Satisfactory | 51--100   |
| Moderate     | 101--200  |
| Poor         | 201--300  |
| Very Poor    | 301--400  |
| Severe       | >400      |

## 13. How to Run

---

```
# Step 1: Install dependencies
pip install -r requirements.txt

# Step 2: Run the full pipeline (Jupyter)
jupyter notebook notebooks/AQI_full_pipeline.ipynb

# Step 3: Launch dashboard
streamlit run dashboard/app.py

# Step 4: Retrain (alternative to notebook)
python scripts/retrain.py

# Step 5: Generate PDF report
pandoc docs/report.md -o docs/AQI_PROJECT_REPORT.pdf --pdf-engine=weasyprint
```